Credit-Scoring-Engine — Chaptered Rebuild Curriculum

Chapter 0 — Problem Framing (before importing the CSV)

a. DS Fundamentals: Supervised binary classification — historical labeled examples (applicant → defaulted or not) predict the label for new applicants. What a target/label is, what a feature is, train vs. production data, and what data leakage means (using info you wouldn't actually have at prediction time). **b. Statistics/Math/Finance:** Population vs. sample (307,511 applicants is a sample, not every borrower ever). Base rate / prior probability — knowing ~8% default rate up front already tells you accuracy will be a bad metric. Core lending vocabulary: loan, annuity, credit amount, goods price, "default." **c. Python:** What a DataFrame/Series is. pd.read_csv(), .shape, .head(), .info(), .dtypes. Nothing about modeling yet.

Chapter 1 — Data Loading & First Inspection

a. DS Fundamentals: What "first inspection" checks for — row/column sanity, ID columns vs. feature columns, what the target column looks like, obvious junk values. **b. Statistics/Math:** Descriptive statistics as your first lens on any column — mean, median, mode, spread (range, std, IQR) — and why you check these before anything else. **c. Python:** .describe(), .nunique(), .isnull().sum(), basic column selection/indexing.

Chapter 2 — EDA: Distributions, Missingness, Imbalance, Correlation

a. DS Fundamentals: Why EDA comes before modeling. What class imbalance means and why it reshapes every later decision. Why domain anomalies (like the DAYS_EMPLOYED sentinel value) only get caught by looking, never by assuming. **b. Statistics/Math:** Skewness and why it decides mean vs. median. Correlation coefficient and correlation is not causation. Empirical probability (observed default rate). Why missing-data severity gets tiered (<10% / 10–50% / >50%) instead of one blanket rule. **c. Python:** matplotlib/seaborn histograms, boxplots, heatmaps. df.corr(), df.isnull().mean().

Chapter 3 — Feature Engineering & Encoding

a. DS Fundamentals: Nominal vs. ordinal vs. continuous vs. discrete data, and why that classification decides the encoding method. Why domain-derived ratios often beat raw columns. What feature leakage would look like here. **b. Statistics/Math/Finance:** What DEBT_TO_INCOME, ANNUITY_TO_INCOME, and CREDIT_TO_GOODS actually mean financially, and why ratios normalize scale better than raw amounts. Log transform for right-skewed variables. **c. Python:** pd.get_dummies (one-hot encoding), manual/target encoding via groupby, np.log1p, basic regex/string cleanup.

Chapter 4 — Train/Validation/Test Split & Imputation

a. DS Fundamentals: Why you split before handling missing values (avoid leaking val/test info into train). Why three sets, not two. What stratification protects against. **b. Statistics/Math:** A random split should preserve the ~8% base rate in every subset — this is sampling reasoning, not an implementation detail. Median imputation reasoning, revisited from Chapter 1-2. **c. Python:** train_test_split, SimpleImputer(strategy='median'), StandardScaler.

Chapter 5 — Baseline Model: Logistic Regression

a. DS Fundamentals: Why you always build a simple, interpretable baseline before a complex model — it's a floor to beat, not a throwaway step. **b. Statistics/Math:** The logistic function / log-odds — how a linear combination of features becomes a 0-1 probability. Reading a coefficient's sign and magnitude. How class_weight='balanced' mathematically reweights the loss function. **c. Python:** LogisticRegression(class_weight='balanced'), reading .coef_.

Chapter 6 — Main Model: LightGBM

a. DS Fundamentals: What gradient boosting is (sequential trees, each correcting the last one's errors) and why it typically beats logistic regression here (nonlinearities, feature interactions, native missing-value handling) — at the cost of interpretability. **b. Statistics/Math:** The scale_pos_weight formula and why it's LightGBM's version of class_weight='balanced'. What early stopping prevents (overfitting) and how to read a validation curve. **c. Python:** lgb.train, lgb.early_stopping, basic params (num_leaves, learning_rate).

Chapter 7 — Evaluation: AUC, Gini, AUC-PR

a. DS Fundamentals: Why accuracy is meaningless on an imbalanced target. What each metric actually answers, and why you look at more than one. **b. Statistics/Math:** ROC curve construction (TPR vs. FPR across thresholds). Gini = 2xAUC - 1. The precision/recall trade-off, and why AUC-PR is harsher under imbalance than AUC-ROC. **c. Python:** roc_auc_score, average_precision_score, roc_curve, precision_recall_curve.

Chapter 8 — Interpretability: SHAP (genuinely not built yet)

a. DS Fundamentals: Why a strong model isn't enough in a regulated domain. Individual (per-applicant) vs. global explanation. **b. Statistics/Math:** Shapley values from game theory — how "contribution" is fairly split across features. Log-odds space vs. probability space in SHAP's output. **c. Python:** shap.TreeExplainer, shap_values, waterfall/summary plots.

Chapter 9 — Dashboard: Streamlit (genuinely not built yet)

a. DS Fundamentals: What a serving interface must reproduce exactly — the same preprocessing pipeline used at training time. This is the single most common real-world bug source (train/serve skew). **b. Statistics/Math/Finance:** Translating a raw probability back into a business decision — threshold choice, risk tiers. **c. Python:** Streamlit basics — widgets, caching, structuring an app file.